Tasca 003 Programació en PL/SQL.

A continuació cal treballar amb una base de dades d'una escola. Primerament hem d'executar un script per crear **l'esquema** a partir de les sentències següents:

```
--Fitxer bbdd_escola.sql

DROP TABLE IF EXISTS notes;
DROP TABLE IF EXISTS alumnes;
DROP TABLE IF EXISTS moduls;
DROP TABLE IF EXISTS auditoria;

CREATE TABLE alumnes (
id serial PRIMARY KEY,
nom varchar(30) NOT NULL,
aprovat boolean DEFAULT false,
matriculats integer DEFAULT 0,
naixement date );

CREATE TABLE moduls (
modul varchar(3) PRIMARY KEY,
descripcio varchar(100));

CREATE TABLE notes (
alumne integer REFERENCES alumnes(id),
modul varchar(3) REFERENCES moduls(modul),
nota numeric(4, 2),
registre timestamp DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (alumne, modul));

CREATE TABLE auditoria (
id serial PRIMARY KEY,
taula varchar(15) NOT NULL,
accio varchar(10) NOT NULL,
usuari varchar(20) NOT NULL,
temps timestamp DEFAULT CURRENT_TIMESTAMP);


INSERT INTO alumnes (nom, naixement) VALUES ('Alex', '1975-12-07');
INSERT INTO alumnes (nom, naixement) VALUES ('Pere', '1981-10-01');
INSERT INTO alumnes (nom, naixement) VALUES ('Anna', '1986-01-07');
INSERT INTO alumnes (nom, naixement) VALUES ('Marta', '1982-05-22');
INSERT INTO alumnes (nom, naixement) VALUES ('Joan', '1995-07-18');

INSERT INTO moduls VALUES ('M07', 'Desenvolupament web en entorn servidor');
INSERT INTO moduls VALUES ('M02', 'Bases de Dades');

INSERT INTO notes (alumne, modul, nota) VALUES (1,'M02', 9.5);
INSERT INTO notes (alumne, modul, nota) VALUES (1,'M07', 8.5);
```

Tasca 003 Programació en PL/SQL.

1. Carregueu l'script **bbdd_escola.sql** amb el client psql. Indiqueu la comanda i afegiu una captura on es vegi el resultat.

```
CREATE SCHEMA bbdd_escola;
SET search_path TO bbdd_escola, "$user", public;

DROP TABLE IF EXISTS notes;
DROP TABLE IF EXISTS alumnes;
DROP TABLE IF EXISTS moduls;
DROP TABLE IF EXISTS auditoria;

CREATE TABLE alumnes (
id serial PRIMARY KEY,
nom varchar(30) NOT NULL,
aprovat boolean DEFAULT false,
matriculats integer DEFAULT 0,
naixement date );
```

utput pane

Data Output | Explain | Messages | History

```
Query returned successfully: one row affected, 198 msec execution time.
```

2. Aquí heu de crear una funció que retorni l'edat de l'alumne. El paràmetre d'entrada ha de ser l'identificador de l'alumne de la taula **alumnes** i és obligatori. La funció retornarà null si el paràmetre és null o no existeix l'alumne. Afegiu també un comentari en bloc dins la funció explicant què fa.

```
CREATE OR REPLACE FUNCTION edat(id INTEGER) RETURNS INTEGER AS $$
DECLARE
edat INTEGER;
BEGIN
SELECT INTO edat EXTRACT(YEAR FROM AGE(naixement)) FROM alumnes WHERE alumnes.id = $1
IF FOUND THEN
        RETURN edat;
ELSE
        RETURN NULL;
END IF;
END
$$ LANGUAGE 'plpgsql'

SELECT edat(id) FROM alumnes
```

utput pane

Data Output | Explain | Messages | History

	edat integer
1	41
2	35
3	31
4	34
5	21

Tasca 003 Programació en PL/SQL.

Mostreu un exemple de la execució de la funció i del resultat. Podeu afegir una captura de pantalla o directament copiar el text del client psql.

```
CREATE OR REPLACE FUNCTION edat(id INTEGER) AS $$
DECLARE
edat INTEGER;
BEGIN
SELECT INTO edat EXTRACT(YEAR FROM AGE(naixement)) FROM alumnes WHERE
alumnes.id = $1;
IF FOUND THEN RETURN edat;
ELSE RETURN NULL;
END IF;
END
$$ LANGUAGE ‘plpgsql’

SELECT edat(id) FROM alumnes
```

Tasca 003 Programació en PL/SQL.

3. Creeu una funció que a partir d'una nota numèrica passada per paràmetre, retorni la nota escalar (Utilitzant l'escala de notes escalars anglosaxones) . El paràmetre d'entrada és obligatori. Cal declarar les escales com constants A, B, C, D, N, i utilitzar una estructura CASE. Un valor d'entrada superior a 10 o inferior a 0 llença una excepció de tipus WARNING indicant la nota incorrecta, i retorna NULL . Per exemple
WARNING: nota incorrecta -1.1

```
c CONSTANT CHAR := 'C';
d CONSTANT CHAR := 'D';
n CONSTANT CHAR := 'N';
BEGIN
CASE
WHEN ($1 >= 0 AND $1 < 5) THEN RETURN d;
WHEN ($1 >= 5 AND $1 < 7) THEN RETURN c;
WHEN ($1 >= 7 AND $1 < 9) THEN RETURN b;
WHEN ($1 >= 9 AND $1 <= 10) THEN RETURN a;
WHEN ($1 < 0 OR $1 > 10) THEN RAISE EXCEPTION 'WARNING: Nota incorrecta = %! No poder
WHEN ($1 IS NULL) THEN RETURN n;
END CASE;
END
$$ LANGUAGE 'plpgsql'

SELECT escalar(nota) FROM notes
```

Output pane

Data Output | Explain | Messages | History

	escalar bpchar
1	A
2	B

Tasca 003 Programació en PL/SQL.

```
d CONSTANT CHAR := 'D';
n CONSTANT CHAR := 'N';
BEGIN
CASE
WHEN ($1 >= 0 AND $1 < 5) THEN RETURN d;
WHEN ($1 >= 5 AND $1 < 7) THEN RETURN c;
WHEN ($1 >= 7 AND $1 < 9) THEN RETURN b;
WHEN ($1 >= 9 AND $1 <= 10) THEN RETURN a;
WHEN ($1 < 0 OR $1 > 10) THEN RAISE EXCEPTION 'WARNING: Nota incorrecta = %! No pode
WHEN ($1 IS NULL) THEN RETURN n;
END CASE;
END
$$ LANGUAGE 'plpgsql'

SELECT escalar(nota) FROM notes
UPDATE notes SET nota = -3 WHERE modul = 'M02'
```

Output pane

Data Output | Explain | Messages | History

```
ERROR:  WARNING: Nota incorrecta = -3.00! No poden ser inferiors a 0 o superiors a 10

********** Error **********

ERROR: WARNING: Nota incorrecta = -3.00! No poden ser inferiors a 0 o superiors a 10
SQL state: P0001
```

Mostreu un exemple de la execució de la funció amb resultat correcte i un altre amb resultat incorrecte. Podeu afegir una captura de pantalla o directament copiar el text del client psql.

```
CREATE OR REPLACE FUNCTION escalar(nota NUMERIC(4,2)) RETURNS CHAR AS $$
DECLARE
a CONSTANT CHAR := ‘A’;
b CONSTANT CHAR := ‘B’;
c CONSTANT CHAR := ‘C’;
d CONSTANT CHAR := ‘D’;
n CONSTANT CHAR := ‘N’;
BEGIN
CASE
WHEN ($1 >= 0 AND $1 < 5) THEN RETURN d;
WHEN ($1 >= 5 AND $1 < 7) THEN RETURN c;
WHEN ($1 >= 7 AND $1 < 9) THEN RETURN b;
WHEN ($1 >= 9 AND $1 <= 10) THEN RETURN a;
WHEN ($1 < 0 AND $1 > 10) THEN RAISE EXCEPTION ‘WARNING: Nota incorrecta = %!
No poden ser inferiors a 0 o superiors a 10’, $1;
WHEN ($1 IS NULL) THEN RETURN n;
```

Tasca 003 Programació en PL/SQL.

4. Creeu una funció que retorni un text amb un llista de les notes d'un alumne indicat per paràmetre. El paràmetre d'entrada és obligatori. Cal retornar null si el paràmetre és null. No s'ha de fer servir cursos, sinó utilitzar FOR ... IN SELECT. Cal Generar l'excepció '**internal_error**' si l'alumne no existeix, capturar aquesta excepció i mostrar una notificació (NOTICE) indicant el missatge **'Alumne no trobat'**. Cal retornar un text diferent si l'alumne no té cap nota. També cal fer servir els caràcters d'escapada necessaris per a donar format al llistat (salts de línia, tabulacions, etc...). Exemples.

```
escola=> SELECT notes_alumne(1);
         notes_alumne
---------------------------------
 Les notes de l'alumne Alex són:
 M10     9.50
 M07     8.50


escola=> SELECT notes_alumne(3);
        notes_alumne
-------------------------------
 L'alumne Anna no té cap nota
```

```
CREATE OR REPLACE FUNCTION llista (identiti INTEGER) RETURNS VARCHAR AS $$
DECLARE
rec RECORD;
nom_alumn VARCHAR;
funciona CONSTANT VARCHAR := ‘’;
BEGIN
IF $1 IS NULL THE RETURN NULL;
ELSE
IF $1 NOT IN (SELECT id FROM alumnes) THEN
RAISE EXCEPTION ‘Internal error:
NOTICE: Alumne no trobat.’;
ELSE
SELECT INTO nom_alumn nom FROM alumnes WHERE id = $1;
RAISE NOTICE ‘Les notes de % son’, nom_alumn;
IF $1 NOT IN (SELECT alumne FROM notes) THEN
RAISE NOTICE ‘% no té nota’, nom_alumn;
RETURN funciona;
END IF;
FOR rec IN SELECT modul, nota FROM notes WHERE $1 IN (SELECT alumne FROM notes)
LOOP
RAISE NOTICE ‘%    %’, rec.modul, rec.nota;
END LOOP;
END IF;
END IF;
RETURN funciona;
END
$$ LANGUAGE ‘plpgsql’

SELECT llista(1)
```

Tasca 003 Programació en PL/SQL.

```
CREATE OR REPLACE FUNCTION llista(identiti INTEGER) RETURNS VARCHAR AS $$
DECLARE
rec RECORD;
nom_alumn VARCHAR;
funciona CONSTANT VARCHAR := '';
BEGIN
IF $1 IS NULL THEN RETURN NULL;
ELSE
        IF $1 NOT IN (SELECT id FROM alumnes) THEN
        RAISE EXCEPTION 'Internal Error:
        NOTICE: Alumne no trobat.';
        ELSE
                SELECT INTO nom_alumn nom FROM alumnes WHERE id = $1;
                RAISE NOTICE 'Les notes de % son:', nom_alumn;
                IF $1 NOT IN (SELECT alumne FROM notes) THEN
                RAISE NOTICE '% no té nota', nom_alumn;
                RETURN funciona;
        END IF;
```

```
        FOR rec IN SELECT modul, nota FROM notes WHERE $1 IN
        (SELECT alumne FROM notes)
        LOOP
                RAISE NOTICE '%    %', rec.modul, rec.nota;
        END LOOP;
        END IF;
END IF;
RETURN funciona;
END
$$ LANGUAGE 'plpgsql'

SELECT llista(1)
```

Output pane

Data Output | Explain | Messages | History

```
NOTICE:  Les notes de Alex son:
NOTICE:  M07    8.50
NOTICE:  M02    9.50
Total query runtime: 47 msec
1 row retrieved.
```

```
SELECT llista(2)
```

Output pane

Data Output | Explain | Messages | His

```
NOTICE:  Les notes de Pere son:
NOTICE:  Pere no té nota
Total query runtime: 32 msec
1 row retrieved.
```

```
SELECT llista(9)
```

Output pane

Data Output | Explain | Messages

```
ERROR:  Internal Error:
NOTICE: Alumne no trobat.
********** Error **********

ERROR: Internal Error:
NOTICE: Alumne no trobat.
SQL state: P0001
```

Tasca 003 Programació en PL/SQL.

5. La taula **notes** conté les notes de cada alumne per a cada mòdul, així cada alumne hauria de tenir tants registres com mòduls a la taula **mòduls**. A partir d'aquestes taules, codifiqueu un procediment **fent servir cursors**, de manera que per a cada parella alumne – mòdul que falti a la taula de **notes** s'insereixi el registre corresponent amb la nota a null.

```
CREATE OR REPLACE FUNCTION notesalum() RETURNS VOID AS $$
DECLARE
curs1 CURSOR FOR SELECT alumnes.id, moduls.modul FROM alumnes, moduls;
idalumn INTEGER;
modalumn VARCHAR;
BEGIN
OPEN curs1;
FETCH curs1 INTO idalumn, modalumn;
WHILE FOUND LOOP
ID (idalumn NOT IN (SELECT alumne FROM notes WHERE modul = modalumn)) THEN
INSERT INTO notes VALUES(idalumn, modalumnm, NULL, current_date);
END IF;
FETCH curs1 INTO idalumn, modalumn;
END LOOP;
END;
$$ LANGUAGE 'plpgsql'

SELECT notesalum();
SELECT * FROM notes
```

```
4   SELECT * FROM notes
5
6   SELECT notesalum();
7
```

put pane

ata Output | Explain | Messages | History

	alumne integer	modul character varying(3)	nota numeric(4,2)	registre timestamp without time zone
1	1	M07	8.50	2017-03-23 16:10:24.236108
2	1	M02	9.50	2017-03-23 16:10:24.236108
3	2	M07		2017-05-25 00:00:00
4	2	M02		2017-05-25 00:00:00
5	3	M07		2017-05-25 00:00:00
6	3	M02		2017-05-25 00:00:00
7	4	M07		2017-05-25 00:00:00
8	4	M02		2017-05-25 00:00:00
9	5	M07		2017-05-25 00:00:00
10	5	M02		2017-05-25 00:00:00

Tasca 003 Programació en PL/SQL.

6. La taula **alumnes** té un camp **aprovat**, que indica si l'alumne té tots els mòduls aprovats. Escriviu un procediment **fent servir cursos** que revisi totes les notes dels alumnes i actualitzi el valor del camp (aprovats = true) per aquells que tenen tots els mòduls aprovats. (Nota >= 5)

   El procediment pot suposar que existeixen totes les notes de tots els mòduls per a tots els alumnes (Resultat d'executar el procediment omplir_notes()).

   En finalitzar el procediment ha de retornar un text indicant el nombre total d'alumnes, el nombre d'aprovats i el percentatge, **és important que el percentatge tingui format**, per exemple

   "Total 10 alumnes, 5 aprovats (50,00%) "

```
CREATE OR REPLACE FUNCTION aprovats() RETURNS VOID AS $$
DECLARE
curs1 CURSOR FOR SELECT alumne, nota FROM notes;
idalumn INTEGER;
notalumn NUMERIC(4,2);
compt INTEGER := 1;
aprov BOOLEAN := true;
numaprovs INTEGER;
percent INTEGER;
BEGIN
OPEN curs1;
FETCH curs1 INTO idalumn, notalumn;
WHILE FOUND LOOP
IF (idalumn != compt) THEN
aprov := true;
compt := compt + 1;
END IF;
IF (idalumn = compt) THEN
IF ((notalumn < 5)OR(notalumn IS NULL)) THEN
aprov := false;
END IF;
IF (aprov = true) THEN
UPDATE alumnes SET aprovat = true WHERE id = idalumn;
END IF;
END IF;
FETCH curs1 INTO idalumn, notalumn;
END LOOP;
SELECT INTO numaprovs COUNT(*) FROM alumnes WHERE aprovat = true;
percent := ((numaprovs::decimal/compt)*100);
RAISE NOTICE 'Total % alumnes, % aprovats (% %%)', compt, numaprovs, percent;
END;
$$ LANGUAGE 'plpgsql'
```

Tasca 003 Programació en PL/SQL.

```
6   SELECT aprovats();
7
8   CREATE OR REPLACE FUNCTION aprovats() RETURNS VOID AS $$
9   DECLARE
10  curs1 CURSOR FOR SELECT alumne, nota FROM notes;
11  idalumn INTEGER;
12  notalumn NUMERIC(4,2);
13  compt INTEGER := 1;
```

utput pane

Data Output | Explain | Messages | History

```
NOTICE:  Total 5 alumnes, 1 aprovats (20 %)
Total query runtime: 32 msec
1 row retrieved.
```

```
1   SELECT * FROM alumnes
2   SELECT * FROM auditoria
3   SELECT * FROM moduls
4   SELECT * FROM notes
5
6   SELECT aprovats();
7
8   CREATE OR REPLACE FUNCTION aprovats() RETURNS VOID AS $$
9   DECLARE
10  curs1 CURSOR FOR SELECT alumne, nota FROM notes;
11  idalumn INTEGER;
12  notalumn NUMERIC(4,2);
13  compt INTEGER := 1;
```

Output pane

Data Output | Explain | Messages | History

	id integer	nom character varying(30)	aprovat boolean	matriculats integer	naixement date
1	2	Pere	f	0	1981-10-01
2	3	Anna	f	0	1986-01-07
3	4	Marta	f	0	1982-05-22
4	5	Joan	f	0	1995-07-18
5	1	Alex	t	0	1975-12-07